

SMART CANTEEN

Technical Architecture & Data Flow Summary

Complete overview of file inputs, server processing, API endpoints, and database models.

AUTHOR: Antigravity Code Assistant

TECHNOLOGY STACK: React 18 / Express.js / MongoDB

DATE GENERATED: June 9, 2026

1. Project Architecture & Directory Layout

Core Framework and Structural Organization

The Smart Canteen project is a modern full-stack web application designed to facilitate online food ordering and status management for campus canteens. The architecture uses a decoupled Client-Server model with shared types for complete type safety.

1.1 Core Architecture Components

- **Frontend Client:** A React Single Page Application (SPA) utilizing React Router 6, TailwindCSS 3 for aesthetics, and Context API for global state management (cart and auth sessions).
- **Backend Server:** An Express.js REST API providing secure routing, payload validation, database CRUD operations, and administrative reports aggregation.
- **Database Layer:** MongoDB serves as the persistent database, mapped using Mongoose ODM to enforce strict schemas for users, menu catalog, and orders.
- **Shared Types:** Shared TypeScript interfaces defined in the shared directory ensure full compile-time validation between client fetch requests and server controllers.

1.2 Core Directory Structure

- **client/App.tsx:** Frontend entry point containing all client-side page routing declarations (user pages and admin portals).
 - **client/context/AppContext.tsx:** Global React context storing active customer session, administrative session, and cart item array.
 - **server/db.ts:** Database connection logic initializing the Mongoose connection to MongoDB.
 - **server/models.ts:** Declares all Mongoose schemas (User, FoodItem, Order, Admin) and seeds default menu items and admin accounts.
 - **server/routes/:** Contains separated router files handling specific API sub-routes (auth, food items, orders, users, reports).
 - **shared/api.ts:** Shared interfaces (e.g. UserType, OrderType, FoodItemType) used by both client and server.
-

2. Database Schema & Data Models

Persistent Collections Structure in MongoDB

The database schema is defined inside `server/models.ts`. It outlines the shape of the documents stored in four main collections:

2.1 User Model (Collection: 'users')

This schema stores credentials and contact details of registered canteen customers. The password is hashed using a SHA-256 algorithm before insertion.

- **name:** String (Required) - Customer's full name
- **email:** String (Required, Unique) - Login identifier
- **password:** String (Required) - SHA-256 hashed password string
- **phone:** String (Required) - Contact number for pick-up notifications
- **timestamps:** `createdAt` & `updatedAt` dates automatically updated by Mongoose

2.2 FoodItem Model (Collection: 'food_items')

Represents individual items available on the menu. Seeded with defaults during database connection.

- **food_name:** String (Required) - Name of the dish
- **category:** String (Required) - Categorization (e.g. Snacks, Lunch, Drinks)
- **price:** Number (Required) - Selling price in local currency
- **image:** String (Required) - Absolute URL of the food item preview image
- **description:** String (Required) - Detailed description of ingredients or size

2.3 Order Model (Collection: 'orders')

Maintains a record of all customer orders. The order items are stored as an embedded sub-document array to freeze pricing and details at checkout time.

- **user_id:** ObjectId (Required) - References the User collection
 - **items:** Sub-document array containing `food_id` (ObjectId ref), `food_name` (String), `quantity` (Number), and `price` (Number)
 - **total_price:** Number (Required) - Total cart value paid
 - **order_status:** String (Enum: Preparing, Ready, Delivered, Cancelled) - Defaults to 'Preparing'
 - **order_date:** Date - Defaults to `Date.now`
-

3. Detailed Data Flows - Customer Side

Trace of User Inputs and Page Transactions

3.1 Customer Registration Flow

- **Input UI File:** client/pages/Register.tsx
- **Captured Data:** Name, Email, Password, Phone via React form inputs.
- **Transmission:** POST request to /api/auth/register containing JSON body.
- **Server Handler:** server/routes/auth.ts -> router.post("/register")
- **Logic & Persistence:** Checks if user email exists. Hashes password with SHA-256. Executes User.create(). Inserts new user record into MongoDB. Responds with status 201.
- **UI Update:** Clears form, displays success toast, and redirects user to /login.

3.2 Customer Login & Session Creation

- **Input UI File:** client/pages/Login.tsx
- **Captured Data:** Email, Password.
- **Transmission:** POST request to /api/auth/login.
- **Server Handler:** server/routes/auth.ts -> router.post("/login")
- **Logic & Persistence:** Hashes input password and queries User.findOne({ email, password: hashedPassword }). If matching document is found, returns user profile details with status 200.
- **State Handling:** Client calls loginUser(userData) in client/context/AppContext.tsx. This sets 'user' state and registers user data inside localStorage.setItem('canteen_user', ...).
- **UI Update:** Redirects user to homepage (/) with logged-in header menu.

3.3 Cart Modification Flow

- **Input UI Files:** client/pages/Menu.tsx (Add to Cart) and client/pages/Cart.tsx (Modify Qty)
- **Captured Action:** Clicking 'Add to Cart' or pressing '+' / '-' buttons.
- **State Handling:** Triggers AppContext functions: addToCart(), updateCartQuantity(), or removeFromCart(). React state 'cart' is updated. State is synced to localStorage.setItem('canteen_cart', ...). No API request is sent at this stage.

4. Detailed Data Flows - Checkout & Admin

Checkout Mechanics and Administrative Actions

4.1 Order Placement & Checkout

- **Input UI File:** client/pages/Cart.tsx
- **Action:** User clicks 'Place Order' button.
- **Transmission:** POST request to /api/orders. Body: { user_id, items: [...], total_price }.
- **Server Handler:** server/routes/orders.ts -> router.post("/")
- **Logic & Persistence:** Validates input arrays. Calls Order.create() which inserts new order in 'orders' collection in MongoDB with status default 'Preparing'. Returns status 201.
- **State Handling:** Client clears cart state and removes 'canteen_cart' from localStorage using clearCart() in AppContext.tsx.
- **UI Update:** Redirects client router to /order/:id where id is the newly created MongoDB order _id.

4.2 Order Cancellation Flow

- **Input UI File:** client/pages/OrderTracking.tsx
- **Action:** User clicks 'Cancel Order' button (available only for 'Preparing' state).
- **Transmission:** PUT request to /api/orders/:id/status. Body: { order_status: 'Cancelled' }.
- **Server Handler:** server/routes/orders.ts -> router.put("/:id/status")
- **Logic & Validation:** Finds order. Checks if current status is 'Preparing'. If preparing, updates DB status to 'Cancelled'. If status has already progressed to 'Ready' or 'Delivered', blocks request and returns error 400.
- **UI Update:** Re-fetches order status to display 'Cancelled' and locks further interactions.

4.3 Admin Authentication Flow

- **Input UI File:** client/pages/admin/AdminLogin.tsx
- **Captured Data:** Username (email), Password.
- **Transmission:** POST request to /api/auth/admin/login.
- **Server Handler:** server/routes/auth.ts -> router.post("/admin/login")
- **Logic & Session:** Queries Admin collection for matching hashed password. On success, client invokes loginAdmin() in AppContext, updating 'admin' state and caching in localStorage.setItem('canteen_admin', ...). Redirects to /admin/dashboard.

5. Detailed Data Flows - Admin Controls

Menu CRUD, Order Fulfillment & Live Analytics

5.1 Admin Menu Item Management (CRUD)

- **Input UI File:** client/pages/admin/ManageFoodItems.tsx
- **Operations:** Create Food Item, Edit Food Item, and Delete Food Item.
- **API Endpoints:** GET /api/food-items (list catalog)
POST /api/food-items (add new item)
PUT /api/food-items/:id (update details)
DELETE /api/food-items/:id (remove item)
- **Server Handler:** server/routes/food.ts
- **Logic & Persistence:** Handles database operations: FoodItem.create(), FoodItem.findByIdAndUpdate(), or FoodItem.findByIdAndDelete(). Writes propagate directly to MongoDB 'food_items' collection.

5.2 Admin Order Fulfillment Workflow

- **Input UI File:** client/pages/admin/ManageOrders.tsx
- **Action:** Admin changes status of order via dropdown (Preparing -> Ready -> Delivered).
- **Transmission:** PUT request to /api/orders/:id/status with { order_status: newStatus }.
- **Server Handler:** server/routes/orders.ts -> router.put("/:id/status")
- **Logic & Persistence:** Updates DB document order status in 'orders' collection. Responds with success JSON.
- **UI Update:** Triggers TanStack query invalidate, reloading admin list and updating customer's tracking page live.

5.3 Admin Dashboard Reports Loading

- **Input UI File:** client/pages/admin/Reports.tsx
- **Transmission:** GET request to /api/reports dispatched on page mount.
- **Server Handler:** server/routes/reports.ts -> router.get("/")
- **Aggregations:** Aggregates values from multiple collections:
 1. Total sales count: Order.countDocuments()
 2. Total customer accounts: User.countDocuments()
 3. Revenue calculation: Sums total_price of non-cancelled orders via aggregation pipeline.
 4. Top 5 popular dishes: Aggregates order item arrays, groups by food_name, sums ordered quantities, and sorts descending.
 5. Sales by food category: Maps ordered food items to categories, calculating revenue per category.
- **UI Update:** Draws pie and bar charts via Recharts representing canteen performance metrics.

6. Quick-Reference Data Mapping Table

End-to-End Flow Summary Directory

User Action / Flow	Input File (Client)	HTTP Path & Method	Database Models & Writes
Registration	pages/Register.tsx	POST /api/auth/register	User.create() (Creates users document)
Login	pages/Login.tsx	POST /api/auth/login	User.findOne() (Verifies password hash)
Browse Menu	pages/Menu.tsx	GET /api/food-items	FoodItem.find() (Fetches food list)
Place Order	pages/Cart.tsx	POST /api/orders	Order.create() (Creates order document)
Track Order	pages/OrderTracking.tsx	GET /api/orders/:id	Order.findById() (Retrieves order data)
Cancel Order	pages/OrderTracking.tsx	PUT /api/orders/:id/status	Order.findByIdAndUpdate() (Updates status)
Admin Login	admin/AdminLogin.tsx	POST /api/auth/admin/login	Admin.findOne() (Verifies admin account)
Food CRUD	admin/ManageFoodItems.tsx	POST/PUT/DELETE /api/food-items/:id	FoodItem.create() FoodItem.findByIdAndUpdate() FoodItem.findByIdAndDelete()
Update Order	admin/ManageOrders.tsx	PUT /api/orders/:id/status	Order.findByIdAndUpdate() (Saves new status)
Analytics Reports	admin/Reports.tsx	GET /api/reports	Order.aggregate() User.countDocuments()

